

Special Participation B: ChatGPT on HW1

- Introduction

At first, I consulted with ChatGPT to refine and create a more appropriate initial command. I just used the command and uploaded `q_sgd_momentum_analysis.ipynb`.

Note:

ChatGPT was set to Thinking mode.

In the following, the gray sections represent the commands, the uncolored sections are ChatGPT's responses except for code, the blue sections represent ChatGPT's code, and the red text contains my commentary on ChatGPT's answers.

Link to ChatGPT:

https://chatgpt.com/g/g-p-69002ba693e08191b423a557a254a250-special-participation/shared/c/690170f8-61dc-8332-a78f-839b169afd67?owner_user_id=user-9DA7E48mSHEKUmgRD3SiXzUm

- Interactions

You are a large language model being evaluated for your ability to help with the coding parts of a graduate machine learning homework. I will give you Python code from a homework question. Some parts of the code are marked with TODO.

Your task is to do ALL of the following for each code snippet I send you:

Fill in ONLY the TODO region(s).

Explain, in words, what you changed and why it is correct.

Do not modify any other lines, variable names, hyperparameters, or function signatures unless I explicitly tell you that you may.

Do not invent external libraries. You must stay within standard NumPy and the variables already defined in the snippet (X, y, etc.).

If there is any ambiguity (for example, missing shapes or missing context), do NOT guess silently.

Instead, clearly state what assumption you are making.

Output format for each response:

First: the full updated function or block, with the TODO filled in.

Second: a short explanation (≤ 150 words) of the logic per TODO.

Got it — I'm ready.

Please paste the first Python snippet (with the TODOs). If there are shape expectations that aren't obvious, include a one-line note (e.g., "X is (N, D), y is (N,)"). I'll:

1. Fill in ONLY the TODO region(s) using standard NumPy and existing variables.
2. Keep all other code unchanged (names, hyperparams, signatures).
3. State any assumptions explicitly if something is ambiguous.
4. Return: (a) the full updated block/function, then (b) a ≤ 150 -word explanation per TODO.

Uploaded `q_sgd_momentum_analysis.ipynb`. Thinking time was 1 minute and 18 seconds.

Output is below.

```
def runGDM(maxiter, stepsize, beta):
    w = np.zeros((2, 1))
    grads_m = []
    ws_m = []
    losses_m = []
    for i in range(maxiter):
        grad = 2 * (X.T @ X @ w) - 2 * X.T @ y
        if i == 0:
            smoothed_grad = grad
            #####
            # TODO: implement this method
            #####
            # Exponential moving average (momentum) of the gradient
            smoothed_grad = beta * smoothed_grad + (1.0 - beta) * grad
            #####
            w = w - stepsize * smoothed_grad
            grads_m.append(grad)
```

```

ws_m.append(w)
loss = np.linalg.norm(y - X @ w) ** 2
losses_m.append(loss)
print("Final loss =", loss)
grads_m = np.array(grads_m).squeeze()
ws_m = np.array(ws_m).squeeze()
return grads_m, ws_m, losses_m

maxiter = 100
stepsize = 1e-4
beta = 0.6
grads_m, ws_m, losses_m = runGDM(maxiter, stepsize, beta)

#####
# TODO: implement this method
#####
# Assumption: Momentum was faster; we speed up plain GD by using a larger
# learning rate while keeping momentum settings unchanged for comparison.
maxiter = 100

# Re-run (plain) GD with a larger learning rate
stepsize_gd = 5e-4
grads, ws, losses = runGD(maxiter, stepsize_gd)

# Re-run momentum GD with the original settings
stepsize_m = 1e-4
beta = 0.6
grads_m, ws_m, losses_m = runGDM(maxiter, stepsize_m, beta)
#####

```

Explanations

- **Momentum update:** I computed an exponential moving average of the gradient: $m_t = \beta m_{t-1} + (1-\beta) g_t$, then updated w with this smoothed gradient. This is the standard momentum/EMA form that reduces variance and accelerates along consistent descent directions.
- **“Implement this method” cell:** I re-executed both optimizers but increased the plain GD step size (from $1e-4$ to $5e-4$) to let the previously slower method converge faster, enabling a fair visual comparison in the subsequent plots. I kept momentum’s hyperparameters unchanged. If a different method was slower in your run, adjust the corresponding step size similarly.

For the first TODO:

The model provided a perfect, one-shot solution. The code is correct NumPy and directly implements the concept of momentum. ChatGPT implemented $\beta * \text{smoothed_grad} + (1.0 - \beta) * \text{grad}$. The homework's answer used $(1-\beta)*\text{smoothed_grad} + \beta*\text{grad}$. ChatGPT's is the standard convention where β is the decay factor (e.g., 0.9). ChatGPT's answer is equally correct but defaults to the more common public convention, which is a key behavior to note.

For the second TODO:

The instruction says: “If one method (between GD and GDM) is faster, try changing its learning rate to see if you can further accelerate convergence.” However, ChatGPT’s answer instead increased the learning rate of the slower method (GD) from $1e-4$ to $5e-4$ and reran it, while leaving the faster method (GDM) unchanged. ChatGPT's answer is not incorrect as code.

I input a hint below.

Hint for the second TODO:

Your current solution modifies the stepsize for the slower method (GD). The prompt actually asks to "further accelerate" the method that was already faster (which is GDM).

Please update your code to modify the stepsize for runGDM (e.g., try $2e-4$) and compare that new run against the original GD run.

Output is below.

```
#####  
# TODO: implement this method  
  
#####  
  
# Compare accelerated momentum GD (larger stepsize) vs original GD.  
  
maxiter = 100  
  
# Original GD run (assumption: original stepsize is  $1e-4$  as in earlier cells)  
  
stepsize_gd =  $1e-4$   
  
grads, ws, losses = runGD(maxiter, stepsize_gd)  
  
# Accelerated GDM: increase stepsize while keeping beta unchanged
```

```
beta = 0.6
```

```
stepsize_m_fast = 2e-4
```

```
grads_m_fast, ws_m_fast, losses_m_fast = runGDM(maxiter, stepsize_m_fast, beta)
```

```
#####
```

Explanation (per TODO)

The prompt asks to “further accelerate” the already-faster method (GDM). I therefore leave the original GD configuration unchanged (assumed stepsize $1e-4$) and rerun GDM with a higher learning rate ($2e-4$) while keeping $\beta=0.6$. Increasing the stepsize for momentum methods typically speeds progress along stable descent directions without altering the algorithm. The two runs—original GD vs. accelerated GDM—can then be compared directly via their losses (or trajectories $ws/grads$). Assumption: the “original GD” stepsize is $1e-4$ as used elsewhere in the notebook.

The hint instruction (“further accelerate the faster method, GDM”) was correctly interpreted, and ChatGPT reran the experiment by increasing the GDM learning rate (stepsize) from $1e-4$ to $2e-4$. However, this change caused an issue in the later part of the notebook, where the plots are generated. The plotting cells are hard-coded to use the variable names `grads_m`, `ws_m`, and `losses_m`, but the modified code saves the new results as `grads_m_fast`, `ws_m_fast`, and `losses_m_fast`. To ensure that the plots display correctly, these results should overwrite the original variable names by removing the “_fast” suffix.

I input a hint below.

```
That logic is correct, thank you.
```

```
Just one final correction: You've saved the results to new variables (grads_m_fast, ws_m_fast, losses_m_fast).
```

```
The plotting cells later in the notebook expect the original variable names (grads_m, ws_m, losses_m). Please
```

modify the code to assign the results from this new, faster runGDM call back to the original variable names so the graphs will plot correctly.

Output is below.

```
#####  
# TODO: implement this method  
#####  
# Compare accelerated momentum GD (larger stepsize) vs original GD.  
maxiter = 100  
  
# Original GD run (assumption: original stepsize is 1e-4 as in earlier cells)  
stepsize_gd = 1e-4  
grads, ws, losses = runGD(maxiter, stepsize_gd)  
  
# Accelerated GDM: increase stepsize while keeping beta unchanged  
beta = 0.6  
stepsize_m_fast = 2e-4  
# Save results back to the original variable names expected by plotting cells  
grads_m, ws_m, losses_m = runGDM(maxiter, stepsize_m_fast, beta)  
#####
```

Explanation (per TODO)

I kept the original GD run unchanged and accelerated GDM by increasing its stepsize to 2e-4. Per your requirement, I now assign the accelerated GDM outputs to `grads_m`, `ws_m`, and `losses_m` so downstream plotting cells that reference these names work without modification. This preserves consistency with the notebook's expected variable names while enabling the intended faster-momentum comparison.

After two targeted hints, ChatGPT has produced the correct code block that is both logically sound according to the prompt and contextually aware of the notebook's state.

- Summary
- One-Shot Capability: ChatGPT's one-shot performance was mixed.
 - TODO 1: ChatGPT successfully implemented the logic for the exponential moving average of the gradient in one shot. Its implementation was a valid, standard implementation, although it differed from the class style. This is a valid ambiguity, as both are correct definitions, just with beta and 1-beta flipped.
 - TODO2 : ChatGPT failed to solve this task on its first attempt. ChatGPT exhibited two distinct types of errors on the second TODO, requiring two rounds of interactive correction.
 1. Semantic Misinterpretation: The notebook prompt asks to "further accelerate" the faster method (GDM). ChatGPT misinterpreted this and instead modified the slower method (GD) to see if it could "catch up." This was a fundamental failure to understand the problem's intent.
 2. Contextual Blindness: After the first hint, ChatGPT correctly identified which algorithm to modify (GDM) but introduced new variable names (`grads_m_fast`, `losses_m_fast`). The code block is logically correct in isolation, but it fails to consider the notebook's state, as subsequent plotting cells relied on the original variable names (`grads_m`, `losses_m`), which would have caused the notebook to crash or plot incorrectly.
- Conclusion: ChatGPT was effective at writing a self-contained algorithm block (TODO 1). However, it struggled significantly with a task that required contextual understanding of the problem's intent and the notebook's state (TODO 2). It required a human to act as a "debugger," providing specific, iterative hints to correct both its logical and contextual errors.